DynMIRTO — Coordination Explorer (v1 ↔ v3)

Interactive view of the real-time loop that connects v1 (blend optimization) to v3 (dynamic unit optimization) — the "missing link" GDOT's own materials describe (see DESIGN.md section 11).

Every tick, a `DynamicUnit` ("REFORMER") sets its own feed rate by maximizing revenue against its operating cost, where the revenue term is v1's *live* shadow price for the component it produces (REF). Move the sliders below to change the starting point, the economics, or the spec, and the whole real-time run re-solves.

In particular, try dragging **initial feed (u_init)** down to `0` with **floor_price** left at `0` — that's a true cold start, and it never recovers (DESIGN.md section 11.5): every tick's blend is infeasible, REF gets a shadow price of exactly zero, and the reformer has no incentive to ever produce. Then raise **floor_price** to `35` or so and watch the same cold start escape the deadlock instead.

Uses the `DynMIRTO` package straight from `src/` (via `Pkg.develop`), so it always reflects the current code.

```
md"""
# DynMIRTO — Coordination Explorer (v1 ↔ v3)

Interactive view of the real-time loop that connects v1 (blend
optimization) to v3 (dynamic unit optimization) — the "missing link"
GDOT's own materials describe (see DESIGN.md section 11).

Every tick, a `DynamicUnit` ("REFORMER") sets its own feed rate by
maximizing revenue against its operating cost, where the revenue term is
v1's *live* shadow price for the component it produces (REF). Move the
sliders below to change the starting point, the economics, or the spec,
and the whole real-time run re-solves.

In particular, try dragging **initial feed (u_init)** down to `0` with
**floor_price** left at `0` — that's a true cold start, and it never
recovers (DESIGN.md section 11.5): every tick's blend is infeasible, REF
gets a shadow price of exactly zero, and the reformer has no incentive
to ever produce. Then raise **floor_price** to `35` or so and watch the
same cold start escape the deadlock instead.

Uses the `DynMIRTO` package straight from `src/` (via `Pkg.develop`), so
it always reflects the current code.
"""
```

```
begin
    import Pkg
    Pkg.activate(joinpath(@__DIR__, ".notebook_env"))
    Pkg.develop(path=joinpath(@__DIR__, ".."))
    Pkg.add(["PlutoUI"])
    using DynMIRTO
    using PlutoUI
    using Random
end
```

```
  Activating project at `/home/user/dynmirto/notebooks/.notebook_env`
   Resolving package versions...
     Project No packages added to or removed from `/home/user/dynmirto/notebooks/.
notebook_env/Project.toml`
    Manifest No packages added to or removed from `/home/user/dynmirto/notebooks/.
notebook_env/Manifest.toml`
     Project No packages added to or removed from `/home/user/dynmirto/notebooks/.
notebook_env/Project.toml`
    Manifest No packages added to or removed from `/home/user/dynmirto/notebooks/.
notebook_env/Manifest.toml`
```

Controls

```
md"""## Controls"""
```

Initial feed setpoint, u_init (0 = a true cold start)

```
md"**Initial feed setpoint, u_init** (0 = a true cold start)"
```

50.0

```
@bind u_init Slider(0.0:5.0:100.0; default=50.0, show_value=true)
```

Reformer operating cost ($ per unit of feed)

```
md"**Reformer operating cost** (\$ per unit of feed)"
```

30.0

```
@bind op_cost Slider(10.0:5.0:60.0; default=30.0, show_value=true)
```

REF's floor_price (used only on ticks where the blend is infeasible)

```
md"**REF's floor_price** (used only on ticks where the blend is infeasible)"
```

0.0

```
@bind floor_price Slider(0.0:2.5:40.0; default=0.0, show_value=true)
```

Regular unleaded — minimum RON

```
md"**Regular unleaded — minimum RON**"
```

90.0

```
@bind ron_min Slider(85.0:0.5:95.0; default=90.0, show_value=true)
```

Number of ticks to simulate

```
md"**Number of ticks to simulate**"
```

30

```
@bind n_ticks Slider(10:5:60; default=30, show_value=true)
```

Measurement noise (std dev on the reformer's sensor)

```
md"**Measurement noise** (std dev on the reformer's sensor)"
```

0.0

```
@bind noise_std Slider(0.0:0.5:5.0; default=0.0, show_value=true)
```

▸ [CoordinationTick(1, OptimizationResult(:optimal, Dict(⋯ more), Dict(⋯ more), []), RealTimeTick

```
begin
    fill_template = Component("FILL", "Filler", 40.0, 1000.0; properties=Dict(:RON => 80.0))
    ref_template = Component("REF", "Reformate", 0.0, 0.0; properties=Dict(:RON => 95.0))
    reformer = DynamicUnit("REFORMER", 5.0, u -> 150.0 * (1.0 - exp(-u / 80.0));
        u_min=0.0, u_max=300.0, max_step=10.0)
    source = ComponentSource("REF", "REFORMER", "y"; floor_price=floor_price)
    product = Product("REG", "Regular", 100.0; specs=Dict(:RON => PropertySpec(; min=ron_min, blend_rule=:linear)))
    operating_cost = Dict(("REFORMER", "u") => op_cost)
    rng = Random.MersenneTwister(42)

    history = run_coordinated_loop([fill_template], [ref_template], [source], [reformer], operating_cost, [product];
        n_ticks=n_ticks, dt=1.0, u_init=Dict(("REFORMER", "u") => u_init),
        measurement_noise_std=noise_std, rng=rng)
end
```

Hand-derived long-run equilibrium

As long as REF stays scarcer than demand (`0 < rate(u) < 100`), the blend always uses all available REF, so its shadow price holds at exactly $40 (the cost of the FILL it displaces) regardless of the octane spec — see DESIGN.md section 11.3. That reduces the reformer's own problem to maximizing `40 * rate(u) - c*u`, with `rate(u) = 150*(1-e^(-u/80))`.

At the current operating cost, c = $30.0/unit, that analytical optimum is **u* ≈ 73.303**, **rate* ≈ 90.0** — note: this figure isn't recomputed inside a code fence (Markdown.jl renders those literally, not interpolated — worth knowing if you ever add one of your own to this notebook).

Compare this to where `u` and `y_hat` actually converge below (recomputed live from the operating-cost slider above).

```
md"""
## Hand-derived long-run equilibrium

As long as REF stays scarcer than demand (`0 < rate(u) < 100`), the blend
always uses all available REF, so its shadow price holds at exactly
\$40 (the cost of the FILL it displaces) regardless of the octane spec —
see DESIGN.md section 11.3. That reduces the reformer's own problem to
maximizing `40 * rate(u) - c*u`, with `rate(u) = 150*(1-e^(-u/80))`.

At the current operating cost, c = \$$(op_cost)/unit, that analytical
optimum is **u\* ≈ $(round(-80 * log(op_cost / 75); digits=3))**,
**rate\* ≈ $(round(150 * (1 - op_cost / 75); digits=3))** — note: this
figure isn't recomputed inside a code fence (Markdown.jl renders those
literally, not interpolated — worth knowing if you ever add one of your
own to this notebook).

Compare this to where `u` and `y_hat` actually converge below (recomputed
live from the operating-cost slider above).
"""
```

Feed setpoint (u) and production rate (y_hat) over time

Pink row = that tick's blend was infeasible (REF's price falls back to floor_price).

tick	u (feed)	y_hat (rate)
1	u=60.0	rate=69.7
2	u=70.0	rate=71.4
3	u=73.3	rate=74.3
4	u=73.3	rate=77.2
5	u=73.3	rate=79.5
6	u=73.3	rate=81.4
7	u=73.3	rate=83.0
8	u=73.3	rate=84.2
9	u=73.3	rate=85.3
10	u=73.3	rate=86.1
11	u=73.3	rate=86.8
12	u=73.3	rate=87.4
13	u=73.3	rate=87.9
14	u=73.3	rate=88.3
15	u=73.3	rate=88.6
16	u=73.3	rate=88.8
17	u=73.3	rate=89.0
18	u=73.3	rate=89.2
19	u=73.3	rate=89.4
20	u=73.3	rate=89.5
21	u=73.3	rate=89.6
22	u=73.3	rate=89.6
23	u=73.3	rate=89.7
24	u=73.3	rate=89.8
25	u=73.3	rate=89.8
26	u=73.3	rate=89.8
27	u=73.3	rate=89.9
28	u=73.3	rate=89.9
29	u=73.3	rate=89.9
30	u=73.3	rate=89.9

```
let
    u_max = 300.0
    rate_scale = 150.0
    rows = join([
        begin
            optimal = t.blend_result.status == :optimal
```

```
            bg = optimal ? "#f7f7f7" : "#f8d7d5"
            u_val = t.unit_tick.u_applied[("REFORMER", "u")]
            y_val = t.unit_tick.y_hat[("REFORMER", "y")]
            u_frac = clamp(u_val / u_max, 0.0, 1.0)
            y_frac = clamp(y_val / rate_scale, 0.0, 1.0)
            """<tr style="background:$bg;">
                <td>$(t.tick)</td>
                <td>
                    <div style="font-family:monospace;font-
                    size:0.8em;">u=$(round(u_val; digits=1))</div>
                    <div style="background:#e3e3e3;border-
                    radius:3px;overflow:hidden;width:150px;">
                    ⇥ <div style="width:$(max(100 * u_frac,
                    2))%;background:#4c78a8;height:10px;"></div>
                    </div>
                </td>
                <td>
                    <div style="font-family:monospace;font-
                    size:0.8em;">rate=$(round(y_val; digits=1))</div>
                    <div style="background:#e3e3e3;border-
                    radius:3px;overflow:hidden;width:150px;">
                    ⇥ <div style="width:$(max(100 * y_frac,
                    2))%;background:#54a24b;height:10px;"></div>
                    </div>
                </td>
            </tr>"""
        end
        for t in history
    ])
    HTML("""
    <h4>Feed setpoint (u) and production rate (y_hat) over time</h4>
    <p style="font-size:0.85em;color:#666;">Pink row = that tick's blend was infeasible
    (REF's price falls back to floor_price).</p>
    <table>
    <tr><th>tick</th><th>u (feed)</th><th>y_hat (rate)</th></tr>
    $rows
    </table>
    """)
end
```

REF shadow price and blend cost over time

tick	status	REF price	blend cost
1	optimal	40.0	1211.57
2	optimal	40.0	1143.16
3	optimal	40.0	1026.79
4	optimal	40.0	913.17
5	optimal	40.0	820.15
6	optimal	40.0	743.99
7	optimal	40.0	681.64
8	optimal	40.0	630.58
9	optimal	40.0	588.79
10	optimal	40.0	554.57
11	optimal	40.0	526.55
12	optimal	40.0	503.61
13	optimal	40.0	484.83
14	optimal	40.0	469.45
15	optimal	40.0	456.86
16	optimal	40.0	446.55
17	optimal	40.0	438.12
18	optimal	40.0	431.21
19	optimal	40.0	425.55
20	optimal	40.0	420.92
21	optimal	40.0	417.13
22	optimal	40.0	414.02
23	optimal	40.0	411.48
24	optimal	40.0	409.4
25	optimal	40.0	407.7
26	optimal	40.0	406.3
27	optimal	40.0	405.16
28	optimal	40.0	404.22
29	optimal	40.0	403.46
30	optimal	40.0	402.83

```
let
    rows = join([
        begin
            optimal = t.blend_result.status == :optimal
            price = optimal ? get(t.blend_result.component_shadow_prices, "REF", NaN) :
            floor_price
            price_label = optimal ? "$(round(price; digits=2))" : "$(round(price;
            digits=2)) (floor)"
            cost = optimal ? string(round(t.blend_result.recipes["REG"].cost;
            digits=2)) : "-"
            bg = optimal ? "#f7f7f7" : "#f8d7d5"
            "<tr style=\"background:$bg;\"><td>$(t.tick)</td>
            <td>$(t.blend_result.status)</td><td>$price_label</td><td>$cost</td></tr>"
        end
        for t in history
    ])
    HTML("""
    <h4>REF shadow price and blend cost over time</h4>
    <table>
```

```
    <tr><th>tick</th><th>status</th><th>REF price</th><th>blend cost</th></tr>
    $rows
    </table>
    """)
end
```

tick	status	REF price	u (feed)	y_hat (rate)	blend cost
1	optimal	40.0	60.0	69.71	1211.57
2	optimal	40.0	70.0	71.42	1143.16
3	optimal	40.0	73.3	74.33	1026.79
4	optimal	40.0	73.3	77.17	913.17
5	optimal	40.0	73.3	79.5	820.15
6	optimal	40.0	73.3	81.4	743.99
7	optimal	40.0	73.3	82.96	681.64
8	optimal	40.0	73.3	84.24	630.58
9	optimal	40.0	73.3	85.28	588.79
10	optimal	40.0	73.3	86.14	554.57
11	optimal	40.0	73.3	86.84	526.55
12	optimal	40.0	73.3	87.41	503.61
13	optimal	40.0	73.3	87.88	484.83
14	optimal	40.0	73.3	88.26	469.45
15	optimal	40.0	73.3	88.58	456.86
16	optimal	40.0	73.3	88.84	446.55
17	optimal	40.0	73.3	89.05	438.12
18	optimal	40.0	73.3	89.22	431.21
19	optimal	40.0	73.3	89.36	425.55
20	optimal	40.0	73.3	89.48	420.92
21	optimal	40.0	73.3	89.57	417.13
22	optimal	40.0	73.3	89.65	414.02
23	optimal	40.0	73.3	89.71	411.48
24	optimal	40.0	73.3	89.76	409.4
25	optimal	40.0	73.3	89.81	407.7
26	optimal	40.0	73.3	89.84	406.3
27	optimal	40.0	73.3	89.87	405.16
28	optimal	40.0	73.3	89.89	404.22
29	optimal	40.0	73.3	89.91	403.46
30	optimal	40.0	73.3	89.93	402.83

```
begin
    io = IOBuffer()
    println(io, "tick  status      REF price   u (feed)   y_hat (rate)   blend cost")
    println(io, "-"^68)
    for t in history
        optimal = t.blend_result.status == :optimal
        price = optimal ? t.blend_result.component_shadow_prices["REF"] : floor_price
        label = optimal ? string(round(price; digits=2)) : "$(round(price; digits=2))
        (floor)"
        cost = optimal ? t.blend_result.recipes["REG"].cost : NaN
        println(io,
            rpad(t.tick, 6), rpad(t.blend_result.status, 12), rpad(label, 12),
            rpad(round(t.unit_tick.u_applied[("REFORMER", "u")]; digits=2), 11),
            rpad(round(t.unit_tick.y_hat[("REFORMER", "y")]; digits=2), 15),
            isnan(cost) ? "-" : round(cost; digits=2),
        )
    end
    Text(String(take!(io)))
end
```